Encapsulation->Access Modifiers

Encapsulation

- Encapsulation allows you to hide specific information and control access to the object's internal state.
- A getter method retrieves an attribute and a setter method changes it.
- It refers to the bundling of data and methods that operate on that data within a single unit, which is called a class in Java.



Access Modifiers in java

- There are two types of modifiers in java: **access modifiers** and **non-access modifiers**.
- The access modifiers in java specifies accessibility (scope) of a data member, method, constructor or class.
- There are 4 types of java access modifiers:

1. private
2. default
3. protected
4. public

- There are many non-access modifiers such as static, abstract, synchronized, native, volatile, transient etc.

All java access modifiers

Visibility	Public	Protected	*Default*	Private
From the same class	Yes	Yes	Yes	Yes
From any class in the same package	Yes	Yes	Yes	No
From a subclass in the same package	Yes	Yes	Yes	No
From a subclass outside the same package	Yes	Yes, *through inheritance*	No	No
From any non-subclass class outside the package	Yes	No	No	No

All java access modifiers

Most Restrictive ⟷ Least Restrictive

Access Modifiers ->	private	Default/no-access	protected	public
Inside class	Y	Y	Y	Y
Same Package Class	N	Y	Y	Y
Same Package Sub-Class	N	Y	Y	Y
Other Package Class	N	N	N	Y
Other Package Sub-Class	N	N	Y	Y

Same rules apply for inner classes too, they are also treated as outer class properties

1. Private access modifier

1. If A class contains private data member and private method and we want to access these private members from outside the class, so there is compile time error.
2. If you make any class constructor private, you cannot create the instance of that class from outside the class.

Example of Private access modifier

```
class  First{
private int  x=20;
private void m1(){System.out.println("Hello world");}
}
public class Main{
 public static void main(String args[]){
    First  obj=new  First();
   System.out.println(obj.x);  //Compile Time Error
   obj.m1();   //Compile Time Error
   }
}
```

Example of Private access modifier

```
class Second{
private Second(){} //private constructor
void m1(){System.out.println("Hello World");}
}
public class Main{
 public static void main(String args[]){
   Second  obj=new  Second();//Compile Time Error
 }
}
```

2.Default access modifier

1. If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package.

Example of Default access modifier

```
//Saved as First.java
package pack;
class A{
  void msg(){System.out.println("Hello");}
}
```

```
//save as Second.java
package mypack;
import pack.*;
class B{
  public static void main(String args[]){
   A obj = new A();//Compile Time Error
   obj.msg();//Compile Time Error
  }
}
```

3.Protected access modifier

1. The **protected access modifier** is accessible within package and outside the package but through inheritance only.
2. The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

Example of Protected access modifier

```
//Saved as First.java
package pack;
Public class First{
  protected void msg()
{System.out.println("Hello");}
}
//save as Second.java
package mypack;
import pack.*;
class Second extends First{
  public static void main(String args[]){
   Second obj = new Second();
   obj.msg();
  }
}
```

4.Public access modifier

1. The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Example of Public access modifier

```
//Saved as First.java
package pack;
Public class First{
  public  void msg(){System.out.println("Hello");}}
//save as Second.java
package mypack;
import pack.*;
class Second{
  public static void main(String args[]){
   First obj = new First();
   obj.msg();
  }
} }
```

Non-access modifiers

- Java provides a number of non-access modifiers to achieve many other functionalities.
- The ***static*** modifier for creating class methods and variables.
- The *final* modifier for finalizing the implementations of classes, methods, and variables.
- The *abstract* modifier for creating abstract classes and methods.
- The *synchronized* and *volatile* modifiers, which are used for threads.